



АКАДЕМИЯ
КОДЕБАЙ

ПРОФЕССИЯ ПЕНТЕСТЕР

Булевы логические операторы, такие как AND (&&) и OR (||) выполняют команды на основе того, была ли предыдущая команда успешной (или вернула True или 0) или неудачной (вернула False или ненулевое значение).

Рассмотрим сначала логический оператор AND (&&), который выполняет команду только в том случае, если предыдущая команда выполнялась (или вернула True или 0) или не выполнялась (вернула False или ненулевое значение).

```
kali@kali:~$ user2=kali
kali@kali:~$ grep $user2 /etc/passwd && echo "$user2 found\!"
kali found!
kali@kali:~$ user2=misha
kali@kali:~$ grep $user2 /etc/passwd && echo "$user2 found\!"
```

В этом примере мы сначала присвоили имя пользователя, которое мы ищем, переменной user2. Далее, мы используем команду grep, чтобы проверить, присутствует ли указанный пользователь в файле /etc/passwd, и, если проверка выполнялась успешно, grep возвращает True и выполняется команда echo. Однако, когда мы пытаемся найти пользователя, который, как мы знаем, не существует в файле /etc/passwd, наша команда echo не выполняется.

При использовании в списке команд оператор OR (||) противоположен AND (&&), он выполняет следующую команду, только если предыдущая команда не выполнялась:

```
kali@kali:~$ echo $user2
misha
kali@kali:~$ grep $user2 /etc/passwd && echo "$user2 found\!" || echo "$user2 not found\!"
misha not found!
```

В приведенном выше примере мы добавили оператор OR (||), за которым следует команда echo. Теперь, когда grep не находит подходящей строки и возвращает значение False, вместо нее выполняется вторая команда echo после оператора OR (||). Эти операторы также можно использовать для сравнения переменных.

Рассмотрим пример:

```
kali@kali:~$ cat > ./and.sh
#!/bin/bash
if [ $USER == 'kali' ] && [ $HOSTNAME == 'kali' ]
then
echo "TRUE"
else
echo "FALSE"
fi
kali@kali:~$ chmod +x ./and.sh
kali@kali:~$ ./and.sh
TRUE
kali@kali:~$ echo $USER && echo $HOSTNAME
kali
kali
```

В этом примере мы использовали AND (&&), и поскольку сравнение переменных было истинно, была выполнена ветвь then.

В test, логический оператор OR (||) используется для проверки одного или нескольких условий, но только одно из них должно быть истинным. Давайте рассмотрим пример:

```
kali@kali:~$ cat > ./or.sh
#!/bin/bash
if [ $USER == 'kali' ] || [ $HOSTNAME == 'pwn' ] ; then
echo "Если одно из условий истинно, выводится эта строка"
else
echo "Вам не повезло!"
fi
kali@kali:~$ chmod +x ./or.sh
kali@kali:~$ ./or.sh
```

Если одно из условий истинно, выводится эта строка

```
kali@kali:~$ echo $USER && echo $HOSTNAME  
kali  
kali
```

В этом примере мы использовали OR (||) для проверки нескольких условий, и поскольку сравнение первой переменной \$USER было истинным, ветвь then выполнялась.